

Assignment 3

End-to-End ML Versioning with Git, DVC & Google Drive

Total Marks: 50

Duration: 2 weeks

Submission Mode: Individual

Deliverables: GitHub repository link, Google Drive DVC remote access, and a PDF report

1. Learning Objectives

By completing this assignment, you will be able to:

- Use advanced Git commands (log variants, diff variants, stash, rebase, reset, mv, rm) confidently in a realistic workflow.
- Structure a TensorFlow ANN project into independent, pipeline-friendly scripts.
- Configure DVC to use Google Drive as remote storage and version large data/model artifacts.
- Author a dvc.yaml pipeline with params.yaml-driven hyperparameters and reproduce it with dvc repro.
- Diagnose and resolve simultaneous code conflicts (Git) and data conflicts (DVC pointer files).

2. Dataset & Problem Statement

Dataset: Fashion-MNIST — 70,000 grayscale 28×28 images across 10 clothing categories (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot). 60,000 images for training, 10,000 for testing. Available directly via `tf.keras.datasets.fashion_mnist.load_data()` — no manual download required.

Problem Statement: Build a fully-connected Artificial Neural Network (ANN — not a CNN) that classifies a Fashion-MNIST image into one of the 10 categories. The target is $\geq 85\%$ test accuracy. The pipeline must be reproducible end-to-end via a single dvc repro call, and every intermediate artifact (raw data, processed data, trained model, metrics) must be versioned through DVC and pushed to a Google Drive remote.

3. Prerequisites & Environment

- Python 3.9+, pip, a virtual environment (venv/conda).
- Packages: tensorflow, dvc, "dvc[gdrive]", pyyaml, scikit-learn, matplotlib.
- A GitHub account and a personal Google account (for Drive).
- Git configured with your name and email (git config --global user.name / user.email).

4. Expected Project Structure

Path	Purpose
src/prepare.py	Downloads Fashion-MNIST, saves raw arrays to data/raw/
src/preprocess.py	Normalizes, splits train/val, saves to data/processed/
src/train.py	Reads params.yaml, builds & trains the ANN, saves models/model.h5
src/evaluate.py	Loads model + test data, writes metrics.json
params.yaml	Central hyperparameters (single source of truth)
dvc.yaml	Pipeline stage definitions (deps, outs, params, metrics)

dvc.lock	Auto-generated lock file after dvc repro
data/raw/, data/processed/	DVC-tracked data folders
models/	DVC-tracked trained model + training history
.gitignore	Excludes DVC-tracked payloads & credentials from Git

Part A — Git Fundamentals & Advanced Commands

Work on a local repository named fashion-ann-pipeline. Push it to a GitHub repo you own. Every sub-task below must be visible in your commit history (do not squash before submission).

- A1.** git init the project, add a README.md and a .gitignore (exclude venv/, __pycache__/, .dvc/cache, *.h5 handled by DVC), and make the first commit on branch main.
- A2.** Create and switch to a branch dev (git checkout -b dev). All feature work for Parts B–D happens on dev with at least 6 incremental commits (one per script/config, not one giant commit).
- A3.** Log variants — run and screenshot the output of: git log --oneline --graph --all, git log --stat -3, git log -p -1, and git log main..dev. In your report, state in 1–2 lines what each variant reveals that the plain git log does not.
- A4.** Diff variants — demonstrate: (a) git diff (unstaged working changes), (b) git diff --staged, and (c) a branch comparison using both git diff main..dev and git diff main...dev. Explain the difference between the two-dot and three-dot branch diff in your report.
- A5.** Stash scenario — while mid-edit on preprocess.py (uncommitted), you need to switch to main to check something. Use git stash, switch branches, switch back, and git stash pop to resume. Show git stash list before popping.
- A6.** Rebase scenario — create a short-lived branch hotfix from main, commit a small fix (e.g., a README typo), merge it into main. Then, from dev, run git rebase main so dev's history sits cleanly on top of the hotfix. Resolve any conflict that appears and show the before/after graph with git log --oneline --graph --all.
- A7.** Reset scenario — make two throwaway commits on a scratch branch. Use git reset --soft HEAD~1 on the first (verify changes remain staged) and git reset --hard HEAD~1 on the second (verify changes are fully discarded). Document the observed difference.
- A8.** Reorganize with git mv (e.g., move loose scripts into src/) and remove an obsolete scratch file with git rm — commit both as tracked history events, not manual delete+re-add.

Part B — Modular TensorFlow ANN Pipeline

Implement four independent scripts inside src/. Each script must run standalone from the command line and must not hard-code hyperparameters — those come from params.yaml (used in Part D).

- B1.** src/prepare.py — Load Fashion-MNIST via keras.datasets, and save the raw train/test images and labels as .npy/.npz files under data/raw/. No hyperparameters needed at this stage.
- B2.** src/preprocess.py — Load the raw arrays, normalize pixel values to [0, 1], and split a validation set out of the training data. Save the resulting train/val/test arrays under data/processed/.
- B3.** src/train.py — Build a Sequential ANN: Flatten → Dense (ReLU) → Dropout → Dense(10, Softmax). Compile with Adam and sparse_categorical_crossentropy. Train on data/processed/, then save the trained model to models/model.h5 and the training history to models/history.csv.
- B4.** src/evaluate.py — Load the trained model and the processed test set. Compute test loss and accuracy, generate a confusion matrix image, and write all metrics to metrics.json at the project root.

Note: You do not need to write extensive boilerplate — keep each script focused on its single responsibility; imports, argument parsing, and yaml loading should be minimal and readable.

Part C — DVC Setup with Google Drive as Remote

- C1.** Install DVC with the Google Drive extra: `pip install "dvc[gdrive]"`.
- C2.** Run `dvc init` inside the Git repository (this must happen after Part A's initial commit, on the dev branch).
- C3.** Create a folder in your personal Google Drive for this project. Copy its folder ID from the browser URL and register it as the default DVC remote: `dvc remote add -d gdrive_storage gdrive://<FOLDER_ID>`.
- C4.** Authenticate by running any `dvc push` — a Google OAuth consent screen opens in your browser. Complete it. Confirm that no credential/token file gets committed to Git (it must be excluded via `.gitignore`).
- C5.** Track your artifacts: `dvc add data/raw data/processed` models. Commit the generated `.dvc` pointer files and updated `.gitignore` to Git, then `dvc push`. Verify the files physically appear inside your Google Drive folder via the browser (screenshot for your report).

Part D — DVC Pipeline (`dvc.yaml` + `params.yaml`)

- D1.** Create `params.yaml` with sections for each stage that needs configuration, e.g. `preprocess` (`test_size`, `seed`) and `train` (`dense_units`, `dropout_rate`, `learning_rate`, `epochs`, `batch_size`). Load it inside the relevant scripts using `yaml.safe_load()`.
- D2.** Author `dvc.yaml` with four stages — `prepare`, `preprocess`, `train`, `evaluate` — wiring each stage's command, its dependencies (script + required input data), its outputs, and, where relevant, which `params.yaml` keys it depends on. The `evaluate` stage's `metrics.json` should be declared so it is trackable as a metric rather than a plain output.
- D3.** Run `dvc repro`. Capture the console log showing all four stages executing in sequence and confirm `dvc.lock` is generated.
- D4.** Change one hyperparameter in `params.yaml` (e.g., increase `dense_units`), and re-run `dvc repro`. In your report, state exactly which stage(s) re-executed and which were skipped, and explain why in 2–3 lines.
- D5.** Commit the updated `params.yaml`, `dvc.yaml`, `dvc.lock`, and `metrics.json` to Git; tag this commit as `v2` (`git tag v2`). Run `dvc push` so the new model/data version reaches your Google Drive remote alongside `v1`.

Part E — Simulated Collaboration & Conflict Resolution

You are working solo, so you will simulate a two-contributor scenario using two branches that both edit overlapping code and regenerate overlapping data — this reliably produces both a Git code conflict and a DVC data conflict for you to resolve.

- E1.** From `main` (after `v1` is merged), create a branch `teammate-sim`. On it, modify `preprocess.py`'s normalization step, regenerate `data/processed/` accordingly, `dvc add` + `dvc push` the new version, and commit the updated `.dvc` pointer + code to this branch.
- E2.** Switch back to `main`. Independently modify `preprocess.py`'s normalization step differently, regenerate `data/processed/` again, and `dvc add` + `dvc push` + commit on `main`.
- E3.** Attempt `git merge teammate-sim` into `main`. You should see a text conflict inside `preprocess.py` and a conflict in the `processed-data .dvc` pointer file (two different hashes for the same path). Screenshot both.
- E4.** Resolve the code conflict by manually reconciling the two normalization approaches in `preprocess.py`. Resolve the data conflict by deciding which `processed-data` version is authoritative (or regenerating a merged version), then run `dvc checkout` to sync your working directory to the resolved pointer.
- E5.** Confirm `dvc status` reports a clean state, complete the merge commit, re-run `dvc repro` to prove the pipeline is reproducible from the merged state, and push the final code (GitHub) and data (Google Drive).

5. Deliverables & Submission

- GitHub repository link with full, unsquashed commit history covering Parts A–E.
- Google Drive DVC remote folder shared with the instructor (or dvc push console output as proof).
- A PDF report (2–4 pages) containing: screenshots/outputs for each Git command in Part A,B,C,D,E along a written explanation o, your final params.yaml and dvc.yaml, the dvc repro logs from D3 and D4 (with your explanation of what reran), the conflict + resolution screenshots from Part E, and a v1-vs-v2 metrics comparison table.
- dvc.lock committed at the final state of the repository.